



hipTsDec:1.5

API Reference

X Platform, IP JPEG XS Decoder TS 1.16.1

28/08/2025

TABLE OF CONTENTS

1	Overview	7
2	commonTypes (1.0)	8
2.1	Type Reference	8
2.1.1	NmosVersionNumber	8
3	component (1.4)	9
3.1	Overview	9
3.2	Type Reference	9
3.2.1	AnyComponent	9
3.2.2	ComponentGroup	9
3.2.3	ComponentLanguage	9
3.2.4	ComponentSelector	9
3.2.5	ComponentType	10
3.2.6	Pid	10
3.2.7	SubtitleFormat	10
4	firewallTypes (1.0)	11
4.1	Type Reference	11
4.1.1	IpAddress	11
4.1.2	PaginatedRequest	11
4.1.3	PaginationInfo	11
4.1.4	SocketAddress	11
4.1.5	UUID	11
5	hipCardSettings (1.3)	12
5.1	Overview	12
5.2	Command Reference	12
5.2.1	GetCardSettings	12
5.2.2	SetCardSettings	12
5.3	Type Reference	12
5.3.1	CardSettings	12
5.3.2	ClockMode	12
5.3.3	GetCardSettings.Request	13
5.3.4	GetCardSettings.Response	13
5.3.5	SetCardSettings.Request	13
5.3.6	SetCardSettings.Response	13
6	hipDedTransport (1.4)	14
6.1	Overview	14
6.2	Type Reference	14
6.2.1	DestinationSettings	14
6.2.2	NetworkParameters	14
6.2.3	OutputUdp	15
6.2.4	OutputUdpCloned	15
6.2.5	OutputUdpSettings	15
6.2.6	VlanPriority	15
7	hipIpInterface (1.2)	17
7.1	Overview	17
7.2	Command Reference	17
7.2.1	GetIpInterfaces	17
7.2.2	SetIpInterfaces	17
7.2.3	DeleteIpInterfaces	17
7.2.4	GetLldpNeighbors	17
7.3	Type Reference	17
7.3.1	DeleteIpInterfaces.Request	17
7.3.2	DeleteIpInterfaces.Response	18
7.3.3	GetIpInterfaces.Request	18
7.3.4	GetIpInterfaces.Response	18
7.3.5	GetLldpNeighbors.Request	18

7.3.6	GetLldpNeighbors.Response	18
7.3.7	GetLldpNeighborsRequest	18
7.3.8	GetLldpNeighborsResponse	18
7.3.9	IpInterface	18
7.3.10	IpInterfaceConfig	19
7.3.11	QuadPortName	19
7.3.12	SetIpInterfaces.Request	19
7.3.13	SetIpInterfaces.Response	19
8	hipNetworkStatus (1.3)	20
8.1	Overview	20
8.2	Command Reference	20
8.2.1	GetNetworkStatus	20
8.2.2	GetSfpStatus	20
8.3	Type Reference	20
8.3.1	GetNetworkStatus.Request	20
8.3.2	GetNetworkStatus.Response	20
8.3.3	GetSfpStatus.Request	20
8.3.4	GetSfpStatus.Response	20
8.3.5	GetSfpStatusRequest	21
8.3.6	GetSfpStatusResponse	21
8.3.7	LinkSpeed	21
8.3.8	NetworkStatus	21
8.3.9	NetworkStatusRequest	21
8.3.10	NetworkStatusResponse	22
9	hipPhysicalPorts (1.1)	23
9.1	Overview	23
9.2	Command Reference	23
9.2.1	GetPhysicalPorts	23
9.2.2	SetPhysicalPorts	23
9.3	Type Reference	23
9.3.1	GetPhysicalPorts.Request	23
9.3.2	GetPhysicalPorts.Response	23
9.3.3	GetPhysicalPortsResponse	23
9.3.4	PhysicalPort	23
9.3.5	SetPhysicalPorts.Request	24
9.3.6	SetPhysicalPorts.Response	24
9.3.7	SetPhysicalPortsRequest	24
10	hipStatus (1.4)	25
10.1	Overview	25
10.2	Command Reference	25
10.2.1	GetEssenceStatus	25
10.2.2	GetThumbnailPath	25
10.3	Type Reference	25
10.3.1	AncStatus	25
10.3.2	AudioStatus	26
10.3.3	DataIdentifier	26
10.3.4	EssenceStatus	26
10.3.5	EssenceStatusRequest	26
10.3.6	EssenceStatusResponse	27
10.3.7	EssenceThumbnailPathRequest	27
10.3.8	EssenceThumbnailPathResponse	27
10.3.9	GetEssenceStatus.Request	27
10.3.10	GetEssenceStatus.Response	27
10.3.11	GetThumbnailPath.Request	27
10.3.12	GetThumbnailPath.Response	27
10.3.13	VideoStatus	28
11	hipTestGenerator (1.0)	29
11.1	Type Reference	29

11.1.1	BitmapBackgroundProperty	29
11.1.2	Color	29
11.1.3	MovingBox	29
11.1.4	TestPattern	29
11.1.5	TextOverlay	29
11.1.6	TextPosition	30
11.1.7	VideoTestGenerator	30
12	hipTsDecStatus (1.2)	31
12.1	Overview	31
12.2	Command Reference	31
12.2.1	GetSourcePids	31
12.2.2	GetTsStatus	31
12.2.3	ClearAllCounters	31
12.3	Type Reference	31
12.3.1	AudioSource	31
12.3.2	ClearAllCounters.Request	31
12.3.3	ClearAllCounters.Response	32
12.3.4	GetSourcePids.Request	32
12.3.5	GetSourcePids.Response	32
12.3.6	GetTsStatus.Request	32
12.3.7	GetTsStatus.Response	32
12.3.8	SourcePids	32
13	hipTsDecoder (1.1)	33
13.1	Overview	33
13.2	Command Reference	33
13.2.1	GetDecoders	33
13.2.2	SetDecoders	33
13.2.3	DeleteDecoders	33
13.3	Type Reference	33
13.3.1	AudioPidConfig	33
13.3.2	Decoder	34
13.3.3	DecoderAncEssence	34
13.3.4	DecoderAudioEssence	34
13.3.5	DecoderEssenceSettings	34
13.3.6	DecoderSourceTs	35
13.3.7	DecoderVideoEssence	35
13.3.8	DeleteDecoders.Request	35
13.3.9	DeleteDecoders.Response	35
13.3.10	GetDecoders.Request	36
13.3.11	GetDecoders.Response	36
13.3.12	SetDecoders.Request	36
13.3.13	SetDecoders.Response	36
14	hipTypes (1.4)	37
14.1	Overview	37
14.2	Type Reference	37
14.2.1	AudioIndex	37
14.2.2	CodingCore	37
14.2.3	FrameRate	38
14.2.4	NumberOfChannels	38
14.2.5	Order	38
14.2.6	ResolutionScan	38
14.2.7	SignalLossAction	39
14.2.8	SignalLossActionAnc	39
14.2.9	SignalLossActionAudio	39
14.2.10	SignalLossActionVideo	39
14.2.11	SocketAddress	40
14.2.12	St2110AudioPacketTime	40
14.2.13	StartChannel	40
14.2.14	TimeReference	40

14.2.15	VideoFormat	40
15	hipValidate (1.0)	42
15.1	Type Reference	42
15.1.1	FloatRange	42
15.1.2	IntRange	42
15.1.3	Result	42
16	linkModes (1.3)	43
16.1	Type Reference	43
16.1.1	IpFecMode	43
16.1.2	IpLinkMode	43
16.1.3	IpLinkSpeed	43
17	lldp (1.1)	44
17.1	Overview	44
17.2	Type Reference	44
17.2.1	LldpMedInventory	44
17.2.2	LldpMode	44
17.2.3	LldpNeighbor	44
17.2.4	LldpNeighborStruct	44
18	lldpProtocols (1.0)	46
19	networkManagerProtocols (1.1)	47
19.1	Overview	47
20	nmosConfig (1.1)	48
20.1	Overview	48
20.2	Type Reference	48
20.2.1	ConnectionType	48
20.2.2	NmosConfig	48
21	nmosLabelConfig (1.0)	49
21.1	Overview	49
21.2	Type Reference	50
21.2.1	EssenceTypeStrings	50
21.2.2	NmosLabelConfig	50
21.2.3	ResourceLabelTemplates	50
22	nmosRegistryConfig (1.0)	51
22.1	Type Reference	51
22.1.1	DnsConfig	51
22.1.2	DnsConfig.mDns	51
22.1.3	DnsSdConfig	51
22.1.4	NmosRegistryConfigStruct	51
22.1.5	RegistryManualConfig	51
23	nmosRegistryStatus (1.0)	53
23.1	Type Reference	53
23.1.1	NmosRegistryStatus	53
24	nmosStatus (1.1)	54
24.1	Overview	54
24.2	Command Reference	54
24.2.1	GetNmosStatus	54
24.3	Type Reference	54
24.3.1	GetNmosStatus.Request	54
24.3.2	GetNmosStatus.Response	54
25	nmosStatusTypes (1.0)	55
25.1	Type Reference	55
25.1.1	NmosStatus	55

26	physicallyPort (1.0)	56
26.1	Type Reference	56
26.1.1	PortName	56
27	sfpConstants (1.3)	57
27.1	Type Reference	57
27.1.1	Identifier	57
27.1.2	Transceiver	57
28	sfpStatus (1.4)	59
28.1	Type Reference	59
28.1.1	SfpDiagnostics	59
28.1.2	SfpStatus	59
29	st2110Types (1.1)	61
29.1	Type Reference	61
29.1.1	St2110Standard	61
30	st2110TypesLabels (1.1)	62
31	tsstatus (1.3)	63
31.1	Overview	63
31.2	Type Reference	63
31.2.1	OutputTransportStreamStatus	63
31.2.2	PidStatus	63
31.2.3	Scrambling	64
31.2.4	ServiceSourceStatus	64
31.2.5	TransportStreamStatus	64

1 Overview

Changelog:

1.5

- **Changed**
 - The imported package nmosConfig is bumped from 1.0 to 1.1.
 - The imported package hipCardSettings is bumped from 1.2 to 1.3.

1.4

- **Changed**
 - nmosStatus from 1.0 to 1.1

1.3

- **Changed**
 - hipIpInterface from 1.1 to 1.2
 - hipNetworkStatus from 1.2 to 1.3
 - hipPhysicalPorts from 1.0 to 1.1
 - networkManagerProtocols from 1.0 to 1.1
 - hipCardSettings from 1.1 to 1.2
- **Added**
 - linkModes added 1.3;
 - nmosConfig added 1.0;

1.2

- **Changed**
 - hipTsDecoder from 1.0 to 1.1
 - hipDecTransport from 1.3 to 1.4
 - hipStatus from 1.3 to 1.4
 - hipTypes from 1.3 to 1.4
 - hipTsDecStatus from 1.1 to 1.2

1.1

- **Changed**
 - hipTsDecStatus from 1.0 to 1.1
 - nmosStatus added 1.0

2 commonTypes (1.0)

2.1 Type Reference

2.1.1 NmosVersionNumber

Generic version number type for NMOS APIs

enum

V1_2

V1_3

3 component (1.4)

3.1 Overview

Changelog:

1.4

- Added
 - str2ComponentType mapping, the inverse of componentType2str.
- Changed
 - Added DVBTMLSUB and JPEG_XS_VIDEO to ComponentType type.

3.2 Type Reference

3.2.1 AnyComponent

empty **struct**

3.2.2 ComponentGroup

ComponentGroup: Grouping of ComponentType, used for filtering

enum

UNDEFINED
AUDIO
VIDEO
SUBTITLE
DATA

3.2.3 ComponentLanguage

string

3.2.4 ComponentSelector

Component selector.

@param pid	Selects a component using a specific (source) PID.
@param language	Selects a component by a three character language string.
@param type	Selects a component by its type.
@param group	Selects a component by group membership.
@param any	Selects any component.

variant

pid	Pid
language	ComponentLanguage
componentType	ComponentType
componentGroup	ComponentGroup
any	AnyComponent

3.2.5 ComponentType

A ComponentType is typically identified by a stream type (see ISO 13818-1 Table 2-29), often distinguished with another header field such as a descriptor tag.

enum

MPEG_VIDEO	
MPEG_AUDIO	
TTX	
DVBSUB	
ECM	
PCR	
PRIVATE	
AC_3	
H264	
VBI	
AAC	
VC_1	
SCS_EMM	
DPI_CUE	
EAC3	
AIT	
HBBTV_CAROUSEL	
DATA_CAROUSEL	
H265	
S302M	
JPEG2000	
S2038	
DVBTTMLSUB	
JPEG_XS_VIDEO	

3.2.6 Pid

int

3.2.7 SubtitleFormat

enum

DVB	
DVB_HARD_OF_HEARING	
EBU	
EBU_HARD_OF_HEARING	

4 firewallTypes (1.0)

4.1 Type Reference

4.1.1 IPAddress

string

4.1.2 PaginatedRequest

Instructs the server to return only a subset of the data found on the server

@param pageSize number of entries returned per request

@param pageNumber what page of the set to returned, determined by the slice [pageSize*pageNumber

struct

pageSize	int
pageNumber	int

4.1.3 PaginationInfo

struct

totalEntries	int
--------------	-----

the total number of entries on the server, before matching

4.1.4 SocketAddress

Tuple with IP address and port

struct

address	IPAddress
port	int

IP address, either IPv4 or IPv6

4.1.5 UUID

string

5 hipCardSettings (1.3)

5.1 Overview

Changelog:

1.3

- **Changed**
 - The imported package `nmosConfig` is bumped from 1.0 to 1.1.

1.2

- **Added**
 - The value `nmosConfig` is added to `cardsettings`.

5.2 Command Reference

5.2.1 GetCardSettings

- message **GetCardSettings.Request**
- message **GetCardSettings.Response**

5.2.2 SetCardSettings

CAUTION: Always use the same UUID as the object already existing on the card.

- message **SetCardSettings.Request**
- message **SetCardSettings.Response**

5.3 Type Reference

5.3.1 CardSettings

- Settings for the operational mode of the card.
-
-

struct

`clockMode`

ClockMode

Type of clock synchronization protocol the card should use

•

`nmos`

optional nmosConfig.NmosConfig

Parameters for accessing NMOS registry and labeling templates

5.3.2 ClockMode

Different types of clock synchronization protocols

enum

PTP	Sets the card to Precision Time Protocol (IEEE 1588)(PTP) time synchronization
LOCAL	Sets the card to Network Time Protocol (NTP) time synchronization or local clock if the card does not get a NTP lock.

5.3.3 GetCardSettings.Request

empty **struct**

5.3.4 GetCardSettings.Response

struct

data **map** from **UUID** to **CardSettings**

5.3.5 SetCardSettings.Request

struct

data **map** from **UUID** to **CardSettings**

5.3.6 SetCardSettings.Response

empty **struct**

6 hipDecTransport (1.4)

6.1 Overview

Changelog:

1.4

- **Added**
 - Added rtpPayload to DestinationSettings.
- **Changed**
 - The imported package hipTypes is bumped from 1.3 to 1.4.

1.3

- **Changed**
 - The imported package hipTypes is bumped from 1.2 to 1.3.

1.2

- **Changed**
 - The imported package hipTypes is bumped from 1.1 to 1.2.

1.1

- **Changed**
 - The imported package hipTypes is bumped from 1.0 to 1.1.

6.2 Type Reference

6.2.1 DestinationSettings

Transport settings for an UDP output

struct

network	NetworkParameters Network parameters
output	OutputUdp UDP output - cloned or single port
rtpPayload	int RTP payload type

6.2.2 NetworkParameters

Network parameters for Output

struct

dscp	int Type of service (TOS in IP header) Also named Differentiated Services Code Point (DSCP)
ttl	int Time to live (TTL in IP header)

vlanPriority

optional **VlanPriority**

Vlan priority. Can only be used if output is on a vlan interface

6.2.3 OutputUdp

UDP output – select cloned output or single output mode

variant

single

OutputUdpSettings

Configuration for the only port used by this output

cloned

OutputUdpCloned

Configuration for port A and port B

6.2.4 OutputUdpCloned

Cloned Configuration for port A and port B

Both interfaces must be configured to run at the same link speed.

struct

sourceA

OutputUdpSettings

Settings on interface D1

sourceB

OutputUdpSettings

Settings on interface D2

6.2.5 OutputUdpSettings

Output settings for UDP outputs

struct

interfaceId

UUID

Interface used

destination

hipTypes.SocketAddress

Unicast or multicast address

sourceAddress

optional string

Overwrite source address in IGMPv3 header

sourcePort

optional int

Overwrite source port in UDP header

6.2.6 VlanPriority

VlanPriority – Set priority related 802.1Q header flags

Value: Traffic types 0 Best effort 1 Background (lowest priority) 2 Excellent effort 3 Critical applications 4 Video, < 100 ms latency and jitter 5 Voice, < 10 ms latency and jitter 6 Internetwork control

struct

pcp

int

Priority code point

dei

bool

Drop eligible indicator

7 hiplInterface (1.2)

7.1 Overview

Changelog:

1.2

- **Changed**
 - The imported package hipPhysicalPorts is bumped from 1.0 to 1.1.

1.1

- **Added**
 - The lldp field is added to the IpInterface object.
 - The GetLldpNeighbors RPC is added

7.2 Command Reference

7.2.1 GetIpInterfaces

- message **GetIpInterfaces.Request**
- message **GetIpInterfaces.Response**

7.2.2 SetIpInterfaces

- message **SetIpInterfaces.Request**
- message **SetIpInterfaces.Response**

7.2.3 DeleteIpInterfaces

- message **DeleteIpInterfaces.Request**
- message **DeleteIpInterfaces.Response**

7.2.4 GetLldpNeighbors

Get LLDP Neighbors.

Returns data about the LLDP neighbor seen per interface. To see data for a specific port, LLDP must be enabled for that Interface.

- message **GetLldpNeighbors.Request**
- message **GetLldpNeighbors.Response**

7.3 Type Reference

7.3.1 DeleteIpInterfaces.Request

struct

ids

list of **UUID**

7.3.2 DeleteIpInterfaces.Response

empty **struct**

7.3.3 GetIpInterfaces.Request

empty **struct**

7.3.4 GetIpInterfaces.Response

struct

data **map** from **UUID** to **IpInterface**

7.3.5 GetLldpNeighbors.Request

GetLldpNeighborsRequest

7.3.6 GetLldpNeighbors.Response

GetLldpNeighborsResponse

7.3.7 GetLldpNeighborsRequest

Get LLDP Neighbors Request

struct

interface

optional list of **UUID**

List of UUIDs uniquely identifying IpInterfaces. Only LLDP Neighbors from the specified IpInterfaces are returned. If no value is set, the request will return neighbors from all ports that have LLDP enabled.

details

bool

If false only chassisID, mgmtIP, and portID will be returned for each LLDP Neighbor. If true all available information is returned.

7.3.8 GetLldpNeighborsResponse

struct

data **map** from **UUID** to **optional lldp.LldpNeighbor**

7.3.9 IpInterface

IP Interface

struct

label

string

User label

physicalPortName	physicalIpPort.PortName The physical port used by the ip interface
enabled	bool Enable/disable the IP interface
vlan	int Vlan number. vlan 0 indicates no vlan in use.
quad	optional QuadPortName Interface number when physical port is set to quad mode (4x10G). Is optional.
ipv4	optional IpInterfaceConfig IPv4 interface configuration. Is optional.
ipv6	optional IpInterfaceConfig IPv4 interface configuration. Is optional.
lldp	optional lldp.LldpMode The LldpMode to set for the port. Leaving this empty indicates LLDP off.

7.3.10 IpInterfaceConfig

IP Interface configuration

struct

ipAddress	string IP address
gateway	string Gateway address of the interface
netmask	int Netmask of the interface in CIDR subnet mask notation e.g /24 = 255.255.255.0

7.3.11 QuadPortName

enum

Q1	
Q2	
Q3	
Q4	

7.3.12 SetIpInterfaces.Request

struct

data	map from UUID to IpInterface
------	---

7.3.13 SetIpInterfaces.Response

empty **struct**

8 hipNetworkStatus (1.3)

8.1 Overview

Changelog:

1.3

- **Changed**
 - The imported package hipPhysicalPorts is bumped from 1.0 to 1.1.

1.2

- **Changed**
 - The imported package sfpStatus is bumped from 1.3 to 1.4.

1.1

- **Added**
 - The imported package sfpStatus 1.3 is added.
 - The command GetSfpStatus is added.

8.2 Command Reference

8.2.1 GetNetworkStatus

- message **GetNetworkStatus.Request**
- message **GetNetworkStatus.Response**

8.2.2 GetSfpStatus

- message **GetSfpStatus.Request**
- message **GetSfpStatus.Response**

8.3 Type Reference

8.3.1 GetNetworkStatus.Request

NetworkStatusRequest

8.3.2 GetNetworkStatus.Response

NetworkStatusResponse

8.3.3 GetSfpStatus.Request

GetSfpStatusRequest

8.3.4 GetSfpStatus.Response

GetSfpStatusResponse

8.3.5 GetSfpStatusRequest

Get SFP Status Request

struct

physicalports

optional list of UUID

List of UUIDs uniquely identifying PhysicalPorts. Only SFP Status from the specified PhysicalPorts are returned. If no value is set, the request will return SFP Status from all ports.

8.3.6 GetSfpStatusResponse

struct

data

map from UUID to **sfpStatus.SfpStatus**

8.3.7 LinkSpeed

LinkSpeed

enum

NO_LINK	No linkspeed
LINK_10_MBPS	10 Mbps
LINK_100_MBPS	100 Mbps
LINK_1_GBPS	1 Gbps
LINK_10_GBPS	10 Gbps
LINK_25_GBPS	25 Gbps
LINK_40_GBPS	40 Gbps

8.3.8 NetworkStatus

NetworkStatus

struct

macAddress	string Interface MAC address.
linkSpeed	LinkSpeed Linkspeed on interface.
rx	optional int Received bitrate on interface (kbps). Only applicable for the IP JPEG XS Encoder.
tx	optional int Transmitted bitrate on interface (kbps). Only applicable for the IP JPEG XS Decoder.

8.3.9 NetworkStatusRequest

NetworkStatusRequest

struct

ids

optional list of **UUID**

Optional list of interface uuids

8.3.10 NetworkStatusResponse

NetworkStatusResponse

struct

data

map from **UUID** to **NetworkStatus**

Map of networkstatus with interface uuid as key

9 hipPhysicalPorts (1.1)

9.1 Overview

Changelog:

1.1

- **Changed**
 - The portMode field in PhysicalPort was changed to linkMode and is using the IpLinkMode type from linkModes 1.3.
- **Added**
 - A fecMode field was added to the PhysicalPort object using the IpFecMode type from linkModes 1.3.

9.2 Command Reference

9.2.1 GetPhysicalPorts

- message **GetPhysicalPorts.Request**
- message **GetPhysicalPorts.Response**

9.2.2 SetPhysicalPorts

- message **SetPhysicalPorts.Request**
- message **SetPhysicalPorts.Response**

9.3 Type Reference

9.3.1 GetPhysicalPorts.Request

empty **struct**

9.3.2 GetPhysicalPorts.Response

GetPhysicalPortsResponse

9.3.3 GetPhysicalPortsResponse

struct

data **map** from **UUID** to **PhysicalPort**

9.3.4 PhysicalPort

Physical Port

struct

name **physicalIpPort.PortName**
Name of the physical port

label	string User label
linkMode	linkModes.IpLinkMode Port mode determines link speed and quad mode
fecMode	optional linkModes.IpFecMode Optional FEC mode
enabled	bool Enable / disable the physical port

9.3.5 SetPhysicalPorts.Request

SetPhysicalPortsRequest

9.3.6 SetPhysicalPorts.Response

empty **struct**

9.3.7 SetPhysicalPortsRequest

struct

data **map** from **UUID** to **PhysicalPort**

10 hipStatus (1.4)

10.1 Overview

Changelog:

1.4

- **Changed**
 - The imported package hipTypes is bumped from 1.3 to 1.4.

1.3

- **Changed**
 - The imported package hipTypes is bumped from 1.2 to 1.3.

1.2

- **Changed**
 - The imported package hipTypes is bumped from 1.1 to 1.2.
 - The field format on type VideoStatus is now optional.

1.1

- **Added**
 - The command GetThumbnailPath is added.
 - The field lineNumber is added to type DataIdentifier.
- **Changed**
 - The imported package hipTypes is bumped from 1.0 to 1.1.

10.2 Command Reference

10.2.1 GetEssenceStatus

Get Encoder essence status

- message **GetEssenceStatus.Request**
- message **GetEssenceStatus.Response**

10.2.2 GetThumbnailPath

Get Video thumbnail path

- message **GetThumbnailPath.Request**
- message **GetThumbnailPath.Response**

10.3 Type Reference

10.3.1 AncStatus

Ancillary Status

struct

dataIdentifiers

list of **DataIdentifier**
List of data identifiers

10.3.2 AudioStatus

Audio Status

struct

totalChannels

int
Total number of of Audio Channels

standard

optional st2110Types.St2110Standard
St2110 audio standard

packetTime

hipTypes.St2110AudioPacketTime
Packet time

10.3.3 DataIdentifier

Data Identifiers

The Data Identifier (DID) and Secondary Data Identifier (SDID) words which together indicate a type of ancillary data.

struct

did

int
Data Identifier word

sdid

int
Secondary Data Identifier word

lineNumber

int
Line number of the received DID/SDID pair

10.3.4 EssenceStatus

Essence Status

Status is reported for all configured essences that have bitrate.

struct

videoStatus

optional VideoStatus
Video status

audioStatus

map from **hipTypes.AudioIndex** to **AudioStatus**
Map of audio statuses

ancStatus

optional AncStatus
Ancillary status

10.3.5 EssenceStatusRequest

Essence Status Request

struct

ids

optional list of UUID

Optional list of Encoder/Decoder UUIDs to request status for. If ids are empty, status for all UUIDs present will be returned.

10.3.6 EssenceStatusResponse

Essence Status Response

struct

data

map from **UUID** to **EssenceStatus**

Map from Encoder/Decoder UUID to the related essence status structure.

10.3.7 EssenceThumbnailPathRequest

Essence Thumbnail Path Request

struct

ids

optional list of UUID

Optional list of Encoder/Decoder UUIDs to request thumbnail path for. If ids are empty, status for all UUIDs present will be returned.

10.3.8 EssenceThumbnailPathResponse

Essence Thumbnail Path Response

struct

data

map from **UUID** to **string**

Map from Encoder/Decoder UUID to the related essence thumbnail path.

10.3.9 GetEssenceStatus.Request

EssenceStatusRequest

10.3.10 GetEssenceStatus.Response

EssenceStatusResponse

10.3.11 GetThumbnailPath.Request

EssenceThumbnailPathRequest

10.3.12 GetThumbnailPath.Response

EssenceThumbnailPathResponse

10.3.13 VideoStatus

Video Status

struct

compressed	bool True if compressed, and false if raw
format	optional hipTypes.VideoFormat Video Resolution and Frame Rate

11 hipTestGenerator (1.0)

11.1 Type Reference

11.1.1 BitmapBackgroundProperty

Specifies the color property of the background in a bitmap.

enum

BLACK	
TRANSPARENT	

11.1.2 Color

A color defined in terms of RGB.

struct

red	int
green	int
blue	int

11.1.3 MovingBox

Specify the properties of a moving box to be used in a test image generator.

struct

enabled	bool
color	optional Color

11.1.4 TestPattern

Pattern to display through generator

enum

BLACK_PICTURE	Display black picture
COLOR_BARS	Display color bars

11.1.5 TextOverlay

Specify the properties of a text overlay to put on top of a test image.

Uses the label of the De/Encoder object as text to display. Text will be truncated after 36 characters

struct

enabled	bool
position	TextPosition
background	BitmapBackgroundProperty

11.1.6 TextPosition

Specify the vertical placement for the text.

enum

TOP
MIDDLE
BOTTOM

11.1.7 VideoTestGenerator

Defines how to generate a test image for a specific port.

struct

forceGenerator	bool
pattern	TestPattern
textOverlay	TextOverlay
movingBox	MovingBox

12 hipTsDecStatus (1.2)

12.1 Overview

Changelog:

1.2

- **Changed**
 - hipTypes from 1.3 to 1.4;

1.1

- **Added**
 - Added type AudioSource
 - Replaced SourcePids.audioPids with AudioStatus

12.2 Command Reference

12.2.1 GetSourcePids

Get Source Pid Status Returns SourcePids per Decoder

- message **GetSourcePids.Request**
- message **GetSourcePids.Response**

12.2.2 GetTsStatus

- message **GetTsStatus.Request**
- message **GetTsStatus.Response**

12.2.3 ClearAllCounters

- message **ClearAllCounters.Request**
- message **ClearAllCounters.Response**

12.3 Type Reference

12.3.1 AudioSource

Decoder TS Audio Source Status

struct

pid	optional int Input Audio Pid
numChannels	optional hipTypes.NumberOfChannels Input number of Audio Channels

12.3.2 ClearAllCounters.Request

empty **struct**

12.3.3 ClearAllCounters.Response

empty **struct**

12.3.4 GetSourcePids.Request

struct

ids **optional list of UUID**

12.3.5 GetSourcePids.Response

struct

data **map from UUID to SourcePids**

12.3.6 GetTsStatus.Request

struct

ids **optional list of UUID**

12.3.7 GetTsStatus.Response

struct

data **map from UUID to tsstatus.TransportStreamStatus**

12.3.8 SourcePids

Decoder TS Audio Pid Status

struct

videoPid

optional int

Input Video Pid

audioPids

map from hipTypes.AudioIndex to map from hipTypes.Order to AudioSource

List Over input audio status

ancPid

optional int

Input Anc Pid

13 hipTsDecoder (1.1)

13.1 Overview

Changelog:

1.1

- Changed
 - hipDecTransport from 1.3 to 1.4;
 - hipTypes from 1.3 to 1.4;

13.2 Command Reference

13.2.1 GetDecoders

Get Decoder objects from database.

- message **GetDecoders.Request**
- message **GetDecoders.Response**

13.2.2 SetDecoders

Update Decoder objects in database.

- message **SetDecoders.Request**
- message **SetDecoders.Response**

13.2.3 DeleteDecoders

Delete Decoder objects from database.

- message **DeleteDecoders.Request**
- message **DeleteDecoders.Response**

13.3 Type Reference

13.3.1 AudioPidConfig

Audio PID Config for essence

Specification which channels to pick from a given pid

struct

pid	optional int Packet Identifier (PID) Audio is extracted from
startChannel	hipTypes.StartChannel Start channel to pick from in pid
numChannels	hipTypes.NumberOfChannels Number of channels in PID

13.3.2 Decoder

Decoder Config

Decoder configuration

struct

enable	bool Enable decoder
label	string Label of decoder
source	DecoderSourceTs TS source struct
coreType	hipTypes.CodingCore Type of Decoder
onSignalLoss	hipTypes.SignalLossAction Action on signal loss

13.3.3 DecoderAncEssence

Decoder TS Anc Essence

Specification for TS ancillary essence source Source has to be of TS ancillary essence type

struct

settings	DecoderEssenceSettings Essence settings
----------	---

13.3.4 DecoderAudioEssence

Decoder TS Audio Essence

Specification for TS audio essence source Source has to be of TS audio essence type

struct

settings	DecoderEssenceSettings Essence settings
standard	st2110Types.St2110Standard ST2110 audio standard
packetTime	hipTypes.St2110AudioPacketTime ST2110 audio packet time
source	map from hipTypes.Order to AudioPidConfig Sources to fill Essence from

13.3.5 DecoderEssenceSettings

Decoder TS Essence Settings

Settings for TS essence source

struct

enable	bool Enable essence
label	string Label of essence
destination	hipDecTransport.DestinationSettings Transport settings

13.3.6 DecoderSourceTs

Decoder TS Source

Specification of TS source

struct

source	UUID Source uuid
videoEssence	DecoderVideoEssence Optional Video essence of source
audioEssences	map from hipTypes.AudioIndex to DecoderAudioEssence List over audio essences
ancEssence	optional DecoderAncEssence Optional ancillary essence

13.3.7 DecoderVideoEssence

Decoder TS Video Essence

Specification for TS video essence source Source has to be of TS video essence type

struct

settings	DecoderEssenceSettings Essence settings
policing	optional hipTypes.VideoFormat Video format policing rules
testGenerator	hipTestGenerator.VideoTestGenerator Video test generator settings

13.3.8 DeleteDecoders.Request

struct

ids	list of UUID
-----	----------------------------

13.3.9 DeleteDecoders.Response

empty **struct**

13.3.10 GetDecoders.Request

empty **struct**

13.3.11 GetDecoders.Response

struct

data **map** from **UUID** to **Decoder**

13.3.12 SetDecoders.Request

struct

data **map** from **UUID** to **Decoder**

13.3.13 SetDecoders.Response

empty **struct**

14 hipTypes (1.4)

14.1 Overview

Changelog:

1.4

- **Added**
 - The enum StartChannel

1.3

- **Added**
 - The value R2160P to ResolutionScan

1.2

- **Removed**
 - The values BLACK_PICTURE, MOVING_BOX and COLOR_BARS are removed from type SignalLossActionVideo.

1.1

- **Added**
 - The type TimeReference is added.

14.2 Type Reference

14.2.1 AudioIndex

Audio Index

enum

AUDIO_1	Audio index 1
AUDIO_2	Audio index 2
AUDIO_3	Audio index 3
AUDIO_4	Audio index 4
AUDIO_5	Audio index 5
AUDIO_6	Audio index 6
AUDIO_7	Audio index 7
AUDIO_8	Audio index 8

14.2.2 CodingCore

Coding Core Type

Type of Coding Core to use.

enum

UHD	Use Core that supports UHD/HD/SD
HD	Use Core that supports HD/SD

14.2.3 FrameRate

Frame Rate

Frame rate of the source.

enum

F24	24 fps
F23_98	23.98 fps
F25	25 fps
F29_97	29.97 fps
F30	30 fps
F47_95	47.95 fps
F48	48 fps
F50	50 fps
F59_94	59.94 fps
F60	60 fps

14.2.4 NumberOfChannels

Number of Channels

enum

CHANNELS_2	2 Mono Channels
CHANNELS_4	4 Mono Channels
CHANNELS_6	6 Mono Channels
CHANNELS_8	8 Mono Channels

14.2.5 Order

Order to put audio channel in.

enum

FIRST	Put audio channels first
SECOND	Put audio channels second
THIRD	Put audio channels third
FOURTH	Put audio channels fourth
FIFTH	Put audio channels fifth
SIXTH	Put audio channels sixth
SEVENTH	Put audio channels seventh
EIGHTH	Put audio channels eighth

14.2.6 ResolutionScan

Resolution and Scan

Resolution and scan mode of the source.

enum

R2160P	2096x2160 progressive
--------	-----------------------

R2KP	2048x1080 progressive
R1080P	1920x1080 progressive
R1080I	1920x1080 interlaced
R720P	1280x720 progressive
R576I	720x576 interlaced
R480I	720x480 interlaced

14.2.7 SignalLossAction

Signal Loss Action

Action taken on loss of bitrate on essences.

struct

videoAction	SignalLossActionVideo Action on video essence
audioAction	SignalLossActionAudio Action on audi essences
ancAction	SignalLossActionAnc Action on ancillary essence

14.2.8 SignalLossActionAnc

Signal Loss Action on ancillary essence

Action taken on loss of bitrate on the ancillary essence.

enum

MUTE	Mutes ancillary essence
TEST_GEN	Generates test ancillary essence

14.2.9 SignalLossActionAudio

Signal Loss Action on audio essences

Action taken on loss of bitrate on the audio essences.

enum

MUTE	Mutes audio essence
TEST_GEN	Generates test audio

14.2.10 SignalLossActionVideo

Signal Loss Action on video essence

Action taken on loss of bitrate on the video essence.

enum

MUTE	Mutes video essence
TEST_GEN	Display picture from test generator

14.2.11 SocketAddress

Socket address

Tuple with IP address and port

struct

address	string IP address, either IPv4 or IPv6
port	int Port number

14.2.12 St2110AudioPacketTime

ST2110 Audio packet time

enum

N_A	No audio packet time
US83_3	83,3 us packet time
US125	125 us packet time
MS1	1 ms packet time

14.2.13 StartChannel

Channel to start on, starts at 1 up to 15 (Only even numbered channel pairs)

enum

CHANNEL_1	Starting at Channel 1
CHANNEL_3	Starting at Channel 3
CHANNEL_5	Starting at Channel 5
CHANNEL_7	Starting at Channel 7
CHANNEL_9	Starting at Channel 9
CHANNEL_11	Starting at Channel 11
CHANNEL_13	Starting at Channel 13
CHANNEL_15	Starting at Channel 15

14.2.14 TimeReference

Time Reference

Protocol used for time reference

enum

NTP	NTP (Network Time Protocol) time source
PTP	PTP (Precision Time Protocol) time source

14.2.15 VideoFormat

Format of the video essence.

struct

resolutionScan

ResolutionScan

Resolution and scan mode of the source

frameRate

FrameRate

Framerate of the source

15 hipValidate (1.0)

15.1 Type Reference

15.1.1 FloatRange

Float Range

struct

min	float Minimum value of the range
max	float Maximum value of the range

15.1.2 IntRange

Integer Range

struct

min	int Minimum value of the range
max	int Maximum value of the range

15.1.3 Result

Validation result

struct

result	bool True if config is valid and false otherwise.
msg	optional string An error message, which may be given if the result is invalid.

16 linkModes (1.3)

16.1 Type Reference

16.1.1 IpFecMode

enum

NO_FEC	
RS_FEC	
FC_FEC	

16.1.2 IpLinkMode

enum

IP_10G	
IP_25G_COPPER	
IP_25G_OPTICAL	
IP_40G	
IP_4x10G	

16.1.3 IpLinkSpeed

enum

IP_10G	
IP_25G	
IP_40G	
IP_4x10G	
NO_LINK	

17 lldp (1.1)

17.1 Overview

```
# Changelog
```

```
## 1.1
```

```
- **Changed**
```

```
- Extracted the structure inside of the `LldpNeighbor` object definition into a new type cal
```

17.2 Type Reference

17.2.1 LldpMedInventory

LLDP-MED Inventory. Appear specific implementation in parentheses.

struct

hwRev	string Hardware Revision
swRev	string Software Revision (Software Version)
fwRev	string Firmware Revision (FPGA Version)
serial	string Serial Number
manufacturer	string Manufacturer ("Appear")
model	string Model ("X10" or "X20")
assetId	string Asset ID (Hardware ID)

17.2.2 LldpMode

Choose either private or public configuration of LLDP frames.

enum

PUBLIC	Only mandatory TLVs and port detail TLVs.
PRIVATE	LLDP-MED inventory in addition to the PUBLIC TLVs.

17.2.3 LldpNeighbor

LldpNeighborStruct

17.2.4 LldpNeighborStruct

The optional variables are only present if `GetLldpNeighborRequest.details` is set to true in the rpc request. The following descriptions are the Appear specific implementations of each LLDP TLV. If neighbors from other manufacturers are detected, TLV values might differ. The `LldpMode` is specified in

parentheses for variables that have different implementations based on the LldpMode set in the neighbor.

struct

chassisId	string MAC address of CTRL on MMI 1, fallback to CTRL on MMI 2 (PRIVATE) or "N/A" (PUBLIC).
mgmtIp	list of string IPv4 address of CTRL on MMI 1 and/or CTRL on MMI 2. Also IPv6 if enabled on the neighbor.
sysDescr	optional string Desktop Heading (PRIVATE) or "Unknown" (PUBLIC)
sysName	optional string Hostname (PRIVATE) or Interface.Port (PUBLIC)
portId	string MAC address of local port
portDescr	optional string Slot, Interface: Port-label
ttl	optional string Time-To-Live, how long an LLDP packet received from that neighbor should be considered valid.
lldpMedInventory	optional LldpMedInventory LLDP-MED Inventory

18 IldpProtocols (1.0)

19 networkManagerProtocols (1.1)

19.1 Overview

Changelog:

1.1

- **Changed**
 - The imported package hipIpInterface is bumped from 1.1 to 1.2.

1.0

- copied from firewall networkManagerProtocol.1.7

20 nmosConfig (1.1)

20.1 Overview

```
# Changelog

## 1.1
- **Added**
  - Added new type `ConnectionType`
  - Added member 'connectionType' to NmosConfig
```

20.2 Type Reference

20.2.1 ConnectionType

Enum to control whether the NMOS traffic is in-band (through line card dataports) or out-of-band (through the MMI).

enum

```
IN_BAND
OUT_OF_BAND
```

20.2.2 NmosConfig

Configuration parameters for setting up NMOS.

struct

connectionType	ConnectionType Configuration to control whether traffic should be in-band or out-of-band
registry	optional nmosRegistryConfig.NmosRegistryConfigStruct Configuration parameters for connecting to the NMOS registry
labelTemplates	optional nmosLabelConfig.NmosLabelConfig Custom NMOS labels configuration. This feature is enabled when the optional is set

21 nmosLabelConfig (1.0)

21.1 Overview

Changelog

1.0

NMOS labelling

NMOS labelling API allows operator to define templates based on auto text options. This templates can be defined for node, device, source, flow, sender and receiver.

Version 1.0 allows six auto text options.

What are the auto text options?

- `${SLOT}` - The slot number of the card in use.
- `${DEV_ID}` - The Device number. **Not supported for Node templates.**
- `${ESS_GR}` - The label of the group where the essences belong to. **Not supported for Node templates.**
- `${ESS}` - The label of the Essence in question. **Not supported for Node and Device templates.**
- `${ESS_T}` - The Essence type: Video, Audio, Ancillary. This auto text option is substituted by what is defined in `EssenceTypeStrings` for each of the essences type. **Not supported for Node and Device templates.**
- `${ESS_T_INDEX}` - The number of the essence belonging to one of the essence type - video, audio, ancillary. **Not supported for Node and Device templates.**

How to use the NMOS Labelling.

- NMOS templates are set in the fields part of `ResourceLabelTemplates`. Unused templates should be set with an empty string.
- Each field in `ResourceLabelTemplates` can have as many auto text options as the operator wants, as long as they are supported for the respective template.
- `EssenceTypeStrings` strings will substitute the auto text option `${ESS_T}` with the defined string for the respective Essence type.

Example of strings to use for the fields in `ResourceLabelTemplates` and `EssenceTypeStrings`

- `nodeTemplate` - ECx210 Decoder ST2110 Sender Slot `${SLOT}`
- `deviceTemplate` - ECx210 Decoder Sender Slot `${SLOT}` ID `${DEV_ID}`
- `sourceTemplate` - ECx210 Slot `${SLOT}` ID `${DEV_ID}`, `${ESS_T}` Source: `${ESS}`, Group: `${ESS_GR}`, Essence Number: `${ESS_T_INDEX}`
- `flowTemplate` - ECx210 Slot `${SLOT}` ID `${DEV_ID}`, `${ESS_T}` Flow: `${ESS}`, Group: `${ESS_GR}`, Essence Number: `${ESS_T_INDEX}`
- `senderTemplate` - ECx210 Slot `${SLOT}` ID `${DEV_ID}`, `${ESS_T}` Sender: `${ESS}`, Group: `${ESS_GR}`, Essence Number: `${ESS_T_INDEX}`
- `receiverTemplate` - ECx210 Slot `${SLOT}` ID `${DEV_ID}`, `${ESS_T}` Receiver: `${ESS}`, Group: `${ESS_GR}`, Essence Number: `${ESS_T_INDEX}`
- `video` - Video
- `audio` - Audio
- `ancillary` - Anc

21.2 Type Reference

21.2.1 EssenceTypeStrings

Labels for each essence type to use when generating custom NMOS labels.

struct

video	string Video essence label
audio	string Audio essence label
ancillary	string Ancillary essence label

21.2.2 NmosLabelConfig

Custom NMOS labels configuration.

struct

resourceTemplates	ResourceLabelTemplates Templates for each NMOS resource
essenceStrings	EssenceTypeStrings Labels to use for each essence type

21.2.3 ResourceLabelTemplates

NMOS label templates for each NMOS resource.

struct

nodeTemplate	string Template string for the node label
deviceTemplate	string Template string for the device label
sourceTemplate	string Template string for the source label
flowTemplate	string Template string for the flow label
senderTemplate	string Template string for the sender label
receiverTemplate	string Template string for the receiver label

22 nmosRegistryConfig (1.0)

22.1 Type Reference

22.1.1 DnsConfig

Method to use for discovery of registry server

variant

mDns	DnsConfig.mDns Use mDNS to reach the registry server
dnsSd	DnsSdConfig Configuration parameters to use DNS-SD for discovery of registry service

22.1.2 DnsConfig.mDns

empty **struct**

22.1.3 DnsSdConfig

Configuration parameters for DNS-SD

struct

dnsServer	firewallTypes.IpAddress IPv4 address of DNS-SD server
search	string Search domain name

22.1.4 NmosRegistryConfigStruct

Configuration parameters for reaching the NMOS registry

struct

registryConfig	RegistryManualConfig Configuration parameters to use in case of failure in mDNS or DNS-SD (secondary/fallback method)
dnsConfig	DnsConfig Configuration parameters for mDNS or DNS-SD (primary method)

22.1.5 RegistryManualConfig

Registry information to be used in case registry server can not be reached by mDNS or DNS-SD.

struct

fallbackEnable	bool Enable use of given information as a fallback
	firewallTypes.IpAddress

registryIpAddr	IPv4 address of NMOS registry
registryPort	int Port number of NMOS registry
registryVersion	commonTypes.NmosVersionNumber Version of registry API

23 nmosRegistryStatus (1.0)

23.1 Type Reference

23.1.1 NmosRegistryStatus

NMOS registry status

struct

ipAddr

optional **firewallTypes.SocketAddress**

Currently used IP address and port number of the registry server. This value may have been obtained from DNS-SD, mDNS, or a manual IP entry.

registered

bool

True if there is an active connection to the registry server.

24 nmosStatus (1.1)

24.1 Overview

Changelog:

1.1

- ****Added****
 - The field 'nodeApi' was added to 'NmosStatus'.
- ****Changed****
 - The type NmosStatus was moved to the file 'nmosStatusTypes'.

24.2 Command Reference

24.2.1 GetNmosStatus

RPC for requesting NMOS status

- message **GetNmosStatus.Request**
- message **GetNmosStatus.Response**

24.3 Type Reference

24.3.1 GetNmosStatus.Request

empty **struct**

24.3.2 GetNmosStatus.Response

nmosStatusTypes.NmosStatus

25 nmosStatusTypes (1.0)

25.1 Type Reference

25.1.1 NmosStatus

NmosStatus Status of the NMOS service

struct

nmosRegistryStatus

nmosRegistryStatus.NmosRegistryStatus

Status of the NMOS registry connection

nodeApi

map from **physicalIpPort.PortName** to **string**

Map from port to the endpoint for NMOS Node API.
Ex: PortName.D1 -> "10.20.10.34:8001", PortName.D2_2 -> "10.20.11.34:8001"

26 physicalIpPort (1.0)

26.1 Type Reference

26.1.1 PortName

enum

CTRL

D1

D2

D3

D4

D1_1

D1_2

D1_3

D1_4

D2_1

D2_2

D2_3

D2_4

D3_1

D3_2

D3_3

D3_4

D4_1

D4_2

D4_3

D4_4

27 sfpConstants (1.3)

27.1 Type Reference

27.1.1 Identifier

Physical Device Identifier Values. (SFF 8472 Rev12.3 spec Table 5-1)

enum

UNKNOWN	Unknown or unspecified
GBIC	GBIC
SOLDERED	Module soldered to motherboard (ex. SFF)
SFP_SFPP	SFP or SFP+
QSFP	QSFP+
QSFP28	QSFP28
NOT_USED	Not used by SFF 8472 Rev12.2.1. These values are maintained in the Transceiver Management section of SFF-8024.
VENDOR	Vendor specific

27.1.2 Transceiver

Transceiver Compliance Codes. (SFF 8472 Rev12.3 spec Table 5-3)

enum

ER_10G	10G Base-ER
LRM_10G	10G Base-LRM
LR_10G	10G Base-LR
SR_10G	10G Base-SR
ACTIVE_CABLE_40G	40G Active cable
LR4_40G	40G Base-LR4
SR4_40G	40G Base-SR4
CR4_40G	40G Base-CR4
PX	BASE-PX
BX10	BASE-BX10
FX_100	100BASE-FX
LX_100	100BASE-LX/LX10
T_1000	1000BASE-T
CX_1000	1000BASE-CX
LX_1000	1000BASE-LX
SX_1000	1000BASE-SX
ELECTRICAL_INTER_ENCLOSURE	Electrical inter enclosure
LONGWAVE_LASER_LL	Longwave laser LL
MEDIUM	Medium distance
LONG_DISTANCE	Long distance
INTERMEDIATE_DISTANCE	Intermediate distance
SHORT_DISTANCE	Short distance
VERY_LONG_DISTANCE	Very long distance
ACTIVE_CABLE	Active Cable
PASSIVE_CABLE	Passive Cable

LONGWAVE_LASER_LC	Longwave laser LC
SHORTWAVE_LASER_w_OFC	Shortwave laser w OFC
SHORTWAVE_LASER_wo_OFC	Shortwave laser w/o OFC
ELECTRICAL_INTRA_ENCLOSURE	Electrical intra enclosure
SINGLE_MODE	Singel mode
MULTIMODE_50um	Multimode 50um
MULTIMODE_50m	Multimode 50m
MULTIMODE_625m	Multimode 62.6m
VIDEO_COAX	Video coax
MINIATURE_COAX	Miniature coax
SHIELDED_TWISTED_PAIR	Shielded twisted pair
TWIN_AXIAL_PAIR	Twin axial pair
M_BYTES_SEC_100	100M bytes per sec
M_BYTES_SEC_200	200M bytes per sec
M_BYTES_SEC_400	400M bytes per sec
M_BYTES_SEC_1600	1600M bytes per sec
M_BYTES_SEC_800	800M bytes per sec
M_BYTES_SEC_1200	1200M bytes per sec
	Extended Specification Compliance. (SFF-8024 Rev 4.8 spec Table 4-4)
AOC_100G_AOC_25GAUI_C2M_BER_5_	100G AOC (Active Optical Cable) or 25GAUI C2M AOC. Providing a worst BER of $5 * 10^{-5}$
SR4_100G_BASE_SR_25G_BASE	100GBASE-SR4 or 25GBASE-SR
LR4_100G_BASE_LR_25G_BASE	100GBASE-LR4 or 25GBASE-LR
ER4_100G_BASE_ER_25G_BASE	100GBASE-ER4 or 25GBASE-ER
SR10_100G_BASE	100GBASE-SR10
CWDM4_100G	100G CWDM4
PSM4_PARALLEL_SMF_100G	100G PSM4 Parallel SMF
ACC_100G_ACC_25GAUI_C2M_BER_5_	100G ACC (Active Copper Cable) or 25GAUI C2M ACC. Providing a worst BER of $5 * 10^{-5}$
CR4_100G_BASE_25G_BASE_CR_CA_2	100GBASE-CR4 or 25GBASE-CR CA-L
CR_25G_BASE_CA_25G_S_50G_BASE_	25GBASE-CR CA-S
CR_25G_BASE_CA_25G_N_50G_BASE_	25GBASE-CR CA-N
ER4_40G_BASE	40GBASE-ER4
SR_4_X_10G_BASE	4 x 10GBASE-SR
PSM4_PARALLEL_SMF_40G	40G PSM4 Parallel SMF
G959_1_PROFILE_P1I1_2D1	G959.1 profile P1I1-2D1 (10709 MBd, 2km, 1310nm SM)
G959_1_PROFILE_P1S1_2D2	G959.1 profile P1S1-2D2 (10709 MBd, 40km, 1550nm SM)
G959_1_PROFILE_P1L1_2D2	G959.1 profile P1L1-2D2 (10709 MBd, 80km, 1550nm SM)
T_10G_BASE	10GBASE-T with SFI electrical interface
CLR4_100G	100G CLR4
AOC_100G_AOC_25GAUI_C2M_BER_10_	100G AOC or 25GAUI C2M AOC. Providing a worst BER of 10^{-12} or below
ACC_100G_ACC_25GAUI_C2M_BER_10_	100G ACC or 25GAUI C2M ACC. Providing a worst BER of 10^{-12} or below
DWDM2_100GE	100GE-DWDM2 (DWDM transceiver using 2 wavelengths on a 1550nm DWDM grid with a reach up to 80km)

28 sfpStatus (1.4)

28.1 Type Reference

28.1.1 SfpDiagnostics

Real Time Diagnostic for SFP/SFP+/QSFP28/QSFP+, with A2h/A0h memory space address in square brackets. (SFF 8472 Rev12.3 spec Table 9-11) (SFF 8636 Rev2.10 spec Table 6-1)

struct

temp	float Internally measured module temperature (°C). [96-97], [22-23]
vcc	float Internally measured supply voltage in transceiver (V). [98-99], [26-27]
txPwr	list of float Measured TX output power (mW). [102-103], [50-57]
rxPwr	list of float Measured RX input power (mW). [104-105], [34-41]

28.1.2 SfpStatus

SFP Status Data Fields, with A0h memory space address in square brackets. (SFF 8472 Rev12.3 spec Table 4-1) QSFP A0h memory space address is specified next to the SFP A0h. (SFF 8636 Rev 2.10 spec Table 6-14)

struct

identifier	sfpConstants.Identifier Type of transceiver (SFP/SFP+ or unknown). [0],[128]
transceiver	list of sfpConstants.Transceiver Code for electronic or optical compatibility [3-10], [131-138]
brNominal	float Nominal signalling rate, units of 100 MBd. [12], [140/222]
vendorName	string SFP vendor name. [20-35], [148-163]
vendorOUI	string SFP vendor IEEE company ID. [37-39], [165-167]
vendorPN	string Part number provided by SFP vendor. [40-55], [168-183]
vendorRev	string Revision level for part number provided by vendor. [56-59], [184-185]
wavelength	string Laser wavelength. [60-61], [186-187]
vendorSN	string Serial number provided by SFP vendor. [68-83], [196-211]
dateCode	string Vendor's manufacturing date code. [84-91], [212-219]

optional SfpDiagnostics

diagnostics

Optional Diagnostics Monitor Data.

29 st2110Types (1.1)

29.1 Type Reference

29.1.1 st2110Standard

The SMPTE ST2110 standards that describe video, audio or ancillary content.

enum

ST2110_20	ST2110 uncompressed video standard
ST2110_22	ST2110 compressed video standard
ST2110_30	ST2110 audio standard
ST2110_31	ST2110 AES3 formatted audio
ST2110_40	ST2110 ancillary standard

30 st2110TypesLabels (1.1)

31 tsstatus (1.3)

31.1 Overview

Changelog:

1.3

- Changed
 - Dependency module component bumped from version 1.1 to 1.4

31.2 Type Reference

31.2.1 OutputTransportStreamStatus

Status for the output TS

struct

services	map from int to ServiceSourceStatus Status for each service in the TS. Map from service ID to status struct.
pids	list of PidStatus Info on all PIDs, both signalled and unsignalled. Map from service PID value to status struct.

31.2.2 PidStatus

PidStatus state of PID

struct

serviceId	list of int List of serviceIds a PID belong to.
componentType	optional component.ComponentType ComponentType of PID
ccErrors	int CC errors on PID
dataRate	bigint Data rate of PID
scrambling	Scrambling Scrambling of PID (none, even or odd)
PCR	bool
present	bool True if signalled pid has rate
language	optional string Language of PID, if applicable (e.g. audio pid)
casId	optional int Id of CAS system, if applicable (e.g. ECM pid)
pidIn	int PID number on input
	optional int

pidOut	PID number on output
--------	----------------------

31.2.3 Scrambling

Scrambling state of PID

enum

NONE	No scrambling
EVEN	Scrambling with even control word
ODD	Scrambling with odd control word

31.2.4 ServiceSourceStatus

Status for an input service

struct

serviceName	optional string Service name
pmtPid	int PMT pid for service
pcrPid	optional int PCR pid for service if present

31.2.5 TransportStreamStatus

Status for the TS

struct

ccErrors	int Total CC error count for this TS.
services	map from int to ServiceSourceStatus Status for each service in the TS. Map from service ID to status struct.
pids	map from int to PidStatus Info on all PIDs, both signalled and unsignalled. Map from service PID value to status struct.
tsId	optional int Transport Stream ID; set if PAT pid present in stream
originalNetworkId	optional int Original Network ID; set if SDT pid present in stream